MMM variable splitter — how to use it

What this is

When a model has too many media variables, we combine several campaigns into one before running it. The model then knows “Meta” but not the five campaigns inside it. Clients want to see the five.

This takes a finished model output file and splits that combined variable back into its parts, on every sheet that names it. It does not re-run the model and it does not change your files: it gives you a new one to download.

It also builds the **Response Curves** and **T ROI curves** sheets when an export does not have them, with the modelling platform’s own formula. It does that whether or not anything is split, so it doubles as the way to add curves to a curve-less export.

Read this first: where your files go

This matters more than anything else in the guide.

The app runs on a server, not on your computer. You open it in the browser and upload two files. While you work, the files are held in memory for your session only. When you press *Run checks* or *Generate*, they are copied to a temporary folder on the server for the length of that run and the folder is deleted when it ends, whatever happens. The result comes back as a download. Nothing is stored, and nothing is shared between users.

Three rules

1 · Use only the app address the firm has approved. Do not run a copy somewhere else, and do not put the code on a different hosting service.

2 · Uploading is handling client material. The model output and import files are client data under the confidentiality agreement. If you are not sure the agreement covers processing them this way, raise it with the client before you upload anything.

3 · Keep the result where you keep other client output. The downloaded file is client data like any model output. Do not save it to a shared or public folder.

Step 1 — Upload the two files

Open the app and drop the files into the two boxes at the top. The **Help** button at the top right opens this guide.

- **Model output file** — the one to be changed (.xlsx).
- **Import file** — the one with the DATA sheet the model was built from.

It must be *the* import file of this model run, not a later or earlier version. If a campaign column has been revised since, the campaigns will not add up to the combined variable and the app stops and tells you which days differ. That has happened on a real pair of files.

The app reads both files and reports how many variables, DATA columns and combined variables it found. If the output file has no curve sheets it says so, and that it will build them.

Step 2 — Choose what to split

Pick one or more combined variables from the list. Composites defined in the model’s *Composite data* sheet are listed first and marked.

Each one gets a table with one row per campaign: **new variable name**, **column with the modelled metric** (the impressions, clicks or GRPs the model was built on) and **column with spend**. There is no count to choose first: add a row with the + under the table, delete one by selecting it.

Pre-filling

- **Composites** are filled in from their formula. A part that is itself a composite stays **one campaign** by default — the level it was combined at — built as the sum of its DATA columns. Tick *Split the composites inside … all the way down* above the table to take it apart instead. A formula that is a rolling average (bma, or cma with an odd width) also fills in the smoothing setting.
- **Variables combined outside the platform** have no formula, so the app searches the DATA columns of the same metric type (impressions with impressions, spend with spend) for a set that adds up exactly to the combined variable, day by day. It fills the set in only if it is the *only* exact one; if two different sets fit (say a 'Total' column next to its campaigns) or none does, it says so and you fill the rows in. The search is a single calculation, not a trial of every combination, so dozens of candidate columns are no problem.
- **Split segments** (*… (Split1)*, *…_Split2*): a combined variable split by period inside the platform. Its campaigns are the combined variable's parts, cut to the segment's period: the app reads the period from the segment itself and sets every campaign column — pre-filled or added by hand — to zero outside it, so you always enter the full columns. The parts come from *Composite data* if the formula is still there, otherwise from the same search, run inside the segment's period. A campaign takes the segment's suffix, e.g. *… (Split1)*, only if its column also ran outside the segment's period — otherwise the cut column is the column itself and keeps its plain name. A segment that ended before the last year gets no ROI curve, but keeps its contributions and *Structure* row.
- **Spend columns** are guessed by swapping the metric part of the name for Inv / Spend / Cost, and only filled in when exactly one column fits.
- Formulas with weights, subtraction or shifts are not plain sums. They are flagged and nothing is filled in, because the tool can only split plain sums. The same goes for a combined variable that mixes metric types (impressions plus clicks): the search does not try those, so fill its rows in by hand.

The most transparent route is still to build combined variables and splits in the import file before modelling: then every part is a column of its own and nothing has to be reconstructed.

Every new variable name must be new: not already a variable in the model, and not a campaign of another variable split in the same run. The app flags a clash at once and the tool refuses to write such a file.

Everything pre-filled is a suggestion. The checks in Step 4 decide whether it is right. Leave a name blank to use the metric column's name.

Per-variable settings

- **Response curves** — how each campaign's curve is drawn. The default is **Native curve per campaign**. The next section explains the four choices.
- **Smoothing before modelling** — only if the combined variable was a rolling average before it went into the model. 1 means none.
- **Diagnose this variable** — for when the checks stop on it. Writes nothing.
- **Overrides** — normally empty. See "Overrides" below.

To add curve sheets without splitting anything, select nothing and go straight to Step 4.

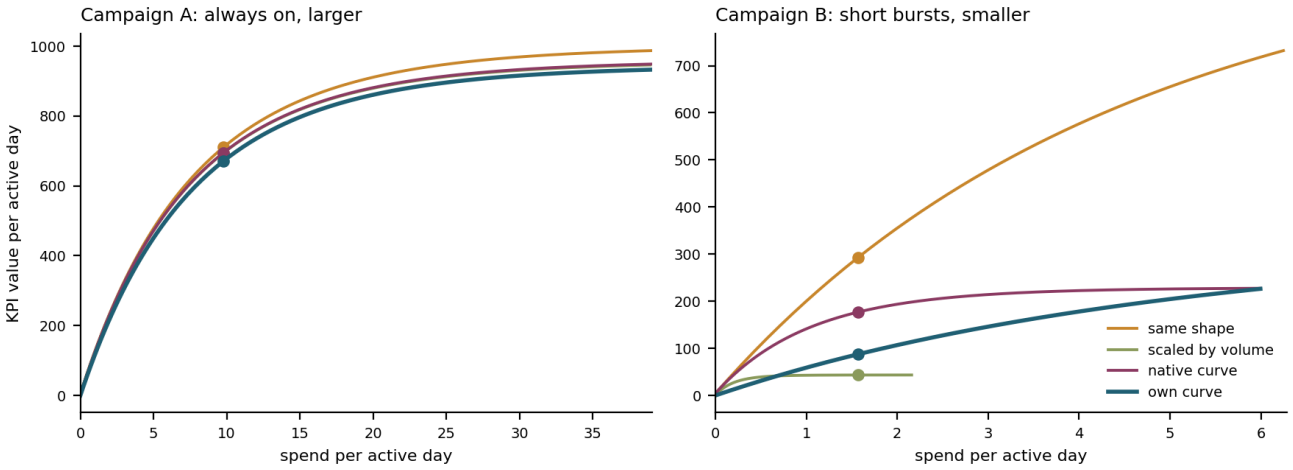
Choosing how the campaign curves are drawn

The model estimated **one** saturation curve for the combined variable. It works on the campaigns' carry-over added together, so one more unit of any campaign lands at the point where the whole combined variable already operates. A per-campaign curve has to be derived from that single curve, and there is more than one way to do it. The app offers four.

Setting	What each campaign's curve is	Adds up to the combined curve?	When to use
Native curve (default)	A curve of the platform's own type, with the campaign's own coefficient and alpha, fitted to the campaign's share of the response when the whole group moves together. Starts at zero.	Yes, within a few percent, when the campaigns move together.	Curves going to the optimiser, and predicting totals. Its parameters are written to the new *Structure* sheet.
Own curve	Built from the model: raise or lower *that* campaign, hold every other campaign at what it actually ran, and read what the model predicts. Starts at zero.	No. The gap is saturation the campaigns share; the run reports its size.	Deciding whether to move money from one campaign to another in the same group: it answers "what would more spend on this campaign alone return?" exactly as the model does.
Same shape, differing by cost	The combined curve itself, with the spend axis set by the campaign's own cost per unit.	No. Each campaign is given the whole combined curve.	Only to show cost efficiency side by side. Never as input to an optimiser.
Scaled by volume	The combined curve shrunk by the campaign's share of contribution.	Yes, exactly.	When the curves must add up to the combined one, and nobody will read a small campaign's curve on its own.

An example

A made-up model, so nothing here is client data. One combined variable holds two campaigns. **A** runs every day and carries most of the volume; **B** runs in short bursts, a fifth of the time, and its media is cheaper per unit. Both feed one diminishing-returns curve.



Dots mark each campaign's current spend. The question that matters is the slope there: how much more KPI one more euro of spend buys. The model's answer comes from raising that campaign's spend a little, alone, and re-running the model's own formula:

Marginal ROI at current spend	Campaign A	Campaign B
What the model says	33.1	47.1
Own curve	33.2	47.1
Native curve (both campaigns move together)	33.2	47.0

Same shape, differing by cost	35.0	152.2 (3.2 times too high)
Scaled by volume	33.0	0.26 (reads as fully saturated)

For the big campaign all four are close, because A *is* most of the combined variable. For the small one they part ways:

- **Same shape** puts B's current spend low on the combined curve, where it is still steep, as if B were running alone. In the model B shares the curve with A, which already sits high on it. So B's next euro is overstated about 3 times, and an optimiser would pour budget into it.
- **Scaled by volume** shrinks the curve by B's small share, so B "saturates" almost at once and looks like a dead end. That is also not what the model says.
- **Own curve** gives 47.1 against the model's 47.1. It is built the same way the model answers the question.
- **Native curve** gives 47.0: its question is a move of the whole group, and here B is so small that A barely changes its answer.

The price of own curves: they do not add up. Here A's and B's own increments come to 95% of what the combined variable delivers. The rest is saturation the two compete for, which belongs to neither. On the rising part of an S curve it can go the other way (more than 100%), because each campaign's carry-over lifts the other. The run prints the figure for every split variable.

How own curves are checked

Before drawing anything, the app rebuilds the combined variable's own curve from the model with the same engine and compares it with the curve in the file. It must match to one part in a million; on our reference exports it matches to about 1e-15. If it does not match, the run stops and says so, rather than drawing campaign curves from an engine that does not reproduce the model. Without a *(COSTS DEF)* sheet the curves are in KPI units (a cost of 1 per period), exactly as the platform draws them.

Native curves

A native curve answers a different question from an own curve: not "what if only this campaign changes?" but "what is this campaign's part when the whole group moves?" That is what usually happens in a budget plan, where every part of a group tends to go up or down together. For each campaign the tool scales the whole group, day by day with carry-over, takes the campaign's share of the response by the same rule that splits the contribution, and fits the platform's own curve to it over the whole modelling period: same curve type, the campaign's own coefficient and alpha, applied to its own series. The last-year ROI curve is then drawn from those parameters, as the platform draws its own. A campaign that ran on its own days, never alongside the others, simply gets the combined variable's coefficient and alpha. The result is a curve the platform itself could draw from one row of *Structure*, and the tool writes that row.

The run prints, per campaign, the fitted curve in platform notation (for example *S Curve (33,6%)*), how closely it fits, and how its slope compares with the own curve within ±20% of current pressure. Per group it prints how closely the native curves add up to the group's response when everything moves together. On a real model the fits were within 4% and the sums within 4%. Where a group is saturated and its campaigns run at the same time, the slope can differ from the own curve by a quarter to two fifths at ±20%: the native curve is then deliberately cautious about moving money between campaigns of that group.

Which curve for what

All four come from the same model, which estimated one curve for the combined variable. They differ in which question they answer.

	Native curve (default)	Own curve	Same shape, differing by cost	Scaled by volume
Strengths	Platform curve type with one alpha per campaign, written to Structure. Starts at zero. Adds up to the group when campaigns move together. Exact for proportional moves.	The model's exact answer for one campaign's next euro with the others unchanged. Built from the model, checked against it to 1e-15.	Simple; same shape as the combined curve; ranks campaigns by cost per unit.	Adds up exactly to the combined curve; simple.
Weaknesses	For moving one campaign alone, the slope is understated above current spend and overstated below it; the gap grows with saturation and overlap.	Curves do not add up: the shared saturation belongs to no campaign (the run reports how much). Not in platform form.	Treats each campaign as if it were the whole group: a small campaign's next euro is overstated (3.2 times in the example). Implies headroom that does not exist.	Splits by contribution share and ignores timing: a small, bursty campaign reads as saturated (0.26 against 47.1 in the example).
Good for	The optimiser; predicting the total of plans that move groups roughly together; reporting; re-entering curves in the platform.	Deciding whether to shift money between campaigns of one group; ranking campaigns by marginal ROI at current spend.	Showing cost efficiency side by side on a chart.	Slides where curves must visibly add up.
Do not use for	Big bets on one campaign against the others in its group; going far beyond the pressure the campaign has ever run at.	Adding curve levels up to predict a total KPI; large simultaneous changes to many campaigns.	Any optimisation or prediction.	Optimisation, or any decision about a small or bursty campaign.
Limitations and how to overcome them	Read the run's 'slope vs own curve' line: above about 20%, check within-group shifts with own curves. For group-wide changes beyond about ±50%, check the total against the combined variable's curve.	For totals, add the shared part back as one fixed amount for the group. After a large reallocation, rebuild the curves around the new plan and evaluate again (two or three rounds).	None worth working around: use a native or own curve instead.	Use the native curve, which is the fitted, timing-aware version of the same idea.

The short version: **native curves** for anything that goes to the optimiser or predicts a total; **own curves** when the question is whether to move money from one campaign to another in the same group.

Step 3 — Settings read from config.py

An expander lists the sheet names and other fixed settings so they are visible rather than hidden. You do not change them in the app. Decay is read as **the share that decays away** each period, which is how our modelling software writes it.

Step 4 — Check

Press **Run checks**. It reads both files, verifies everything and writes nothing. If several people press it at the same moment, runs wait their turn. You get a tick, a cross or a warning against each finding. Read these:

- **Transform check** — rebuilds the combined variable's contribution from its decay, lag, curve and coefficient, and must match the model's own figures to one part in a million. "relative error 4.3e-16" means a match to the 16th decimal.
- **Recency decay** — for decay written like *R 10,0% (6)*, the line "recovered the exact 7-tap kernel" means the tool solved the platform's exact carry-over weights from the model's figures. They are not a plain geometric decay, and treating them as one used to fail the check.
- **The three number pairs** — contribution, spend and raw, before and after. They must match.
- **Curve lines** — the own-curve check ("rebuilt from the model to 1e-15") and how much of the combined response the campaigns' own increments add up to.
- Anything marked with a warning — usually informational, but read it.

If you change anything after the checks have run, the app asks you to run them again. The file can only be generated from settings that were checked.

Step 5 — Generate and download

Name the file and press **Generate the file**, then **Download**. The run takes a few minutes on a large model, most of it writing the workbook. You can also download the settings as a small config file, a record of exactly how the split was made.

When the output file has no curve sheets

Newer exports may not include *Response Curves* and *T ROI curves*. The app then builds both, for every paid-media variable (names starting M-) that has spend in the last year (365 days; on a weekly model the last 52 weeks, the final week included), with the platform's own formula: the same grid of 351 points up to twice the window maximum, the same Average, Maximum and Diminishing Point markers, and the same layout. Checked against an export that has the platform's own sheets, every value matched to within 2e-11.

- A variable whose decay, lag and curve do not rebuild its own contribution exactly gets no curve, with a warning, rather than a curve from parameters that are not the model's.
- Building needs *(SPENDS DEF)* in the output file. Without *(COSTS DEF)* the curves are in KPI units, as the platform draws them in that case. Weekly models work the same way; checked against a weekly export to 2e-12.
- With nothing selected to split, the run only adds the two sheets and every other sheet stays exactly as it was.

Overrides — and why they are not a way to skip a check

Sometimes a check fails for a reason someone has already looked into. The overrides record that, and they are deliberately awkward.

- **You write a reason, and it is printed in the output.** Anyone reading the result sees both the override and the justification.
- **The tool measures whether your reason holds.** It calculates the effect and refuses if it is too large.
- **They apply to one named variable**, inside that variable's own section, so a typo cannot silently apply nothing.

There are three: accept a failed transform check; treat the combined figure as authoritative on a few hand-adjusted days; and accept that the combined Raw series is wrong and the campaign columns are right. Look before you override: a failed transform check already prints how far the split would move, and each variable in step 2 has **Diagnose this variable** with two buttons, *Which columns are missing?* and *Was it smoothed before modelling?*, for when the campaigns do not add up.

The old rank-based transform check scored about 0.97–0.98 on S-curve variables whose settings are in fact exactly right, and some projects overrode it for that reason. The check is now exact, so those overrides should no longer be needed. If one still is, find the cause.

When it stops instead of running

The tool refuses to write a half-correct file. If a check fails you get nothing, plus an explanation of what it found: which variable, which days, by how much. Every stop so far has been a real thing worth knowing about, including an import file that did not match the model run.

Do not work around it. If the campaigns do not add up, try *Diagnose this variable* first. Otherwise send the message to whoever maintains the tool rather than editing the Excel until the error goes away. If the app says the tool hit an *unexpected error*, that is a fault in the tool rather than something about your files. Send that on too.

What it changes

Sheet	What happens
T structure	1 row becomes one per campaign; ROI and CPU recomputed with the file's own definitions
Structure	kept as *Structure (original)*; a new *Structure* in the same format has one row per campaign in place of the combined variable: native coefficient, curve and alpha, fitted over the whole modelling period (so a campaign that has not run in the last year still gets them, though no ROI curve); normalized and standardized coefficients from the campaign's contribution (the platform's own definitions); the combined variable's decay, lag and category; the campaign's ROI and CPU. SE, CI, t-Stat and p-Value stay empty: the model never estimated the campaigns. Standardized coefficients do not add up to the combined one: campaigns that run at different times each vary more than their sum does.
T datasheet variables	that variable's daily rows become one set per campaign
(SPENDS DEF)	1 column becomes one per campaign
Individual contributions	1 column becomes one per campaign
Individual weekly contributions	1 column becomes one per campaign, summed from the same daily split
Response Curves	1 channel block becomes one per campaign; built from the model if the export has none
T ROI curves	1 block becomes one per campaign; built if missing
Everything else	untouched — not even rewritten

A campaign that has not run in the last 12 months gets its spend and contribution but no curve: what it did is history, what it would return tomorrow cannot be answered from a year of no activity.

What the split cannot tell you

The model estimated one effect and one saturation curve for the combined variable. It never compared the campaigns inside it. So differences between the new campaigns come from volume, timing and cost, not from the model judging their quality. A campaign that looks efficient bought attention more cheaply or ran when the curve was less saturated.

If the question is whether a sale click is genuinely worth more than a brand click, no arrangement of this data answers it. That needs the campaigns modelled separately. **Say this before showing anyone a chart, not after.**

For whoever maintains it

The app is six files kept together: `app.py` (the interface), `splitter_service.py` (uploads, run isolation, pre-fill), `mmm_splitter.py` (the tool itself, edited directly), `config.py`, `requirements.txt` and `.streamlit/config.toml`. The command line still works the same way:

```
python mmm_splitter.py split OUTPUT.xlsx IMPORT.xlsx --apply
```

In `config.py`, `CURVE_MODE` defaults to `native_curve`, `BUILD_MISSING_CURVES` to True and `CURVE_STEPS` to 350. Per-job overrides in config.py are ignored by the app, which asks for them per variable instead.

Quick reference

Start it	open the approved app address
Files needed	the model output file and the import file it was built from
Where your files go	the app server, for your session only; deleted after each run
Does it change my files	no — you download a new one
Curve setting	Native curve; own curve for moves within a group
Help	the Help button at the top right of the app opens this guide
No curve sheets in the export	built automatically; select nothing to only add them
Overriding a check	only after running the diagnostic, always with a reason
It stopped	read the message, send it on, do not work around it